

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Rhushabh Vaghela	India	rhushabhvaghela@gmail.com	
Robert Nkuna	South Africa	nkunarobert60@gmail.com	
Israel Olayinka Shodiya	Nigeria	shodiyaisraelolayinka@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an “X” above).

Team member 1	Rhushabh Vaghela
Team member 2	Robert Nkuna
Team member 3	Shodiya Israel

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.

Note: You may be required to provide proof of your outreach to non-contributing members upon request.

N/A

Report

Issue 1: Optimizing Hyperparameters

(Author: Rhushabh Vaghela | Reviewer: Israel Olayinka Shodiya)

Technical

Description of Hyperparameters:

When building quantitative models, it is important to distinguish clearly between internal parameters and hyperparameters. The algorithm learns internal parameters—like the weights in a neural network—entirely on its own during training. Hyperparameters, on the other hand, are the overarching structural rules that the analyst has to set manually before training even starts (Yu and Zhu 2). Think of things like the learning rate (η) that controls gradient descent steps, regularization penalties (C), or tree depth limits. Leaving these settings at software defaults often leads to poor out-of-sample performance.

Optimization Search Methods:

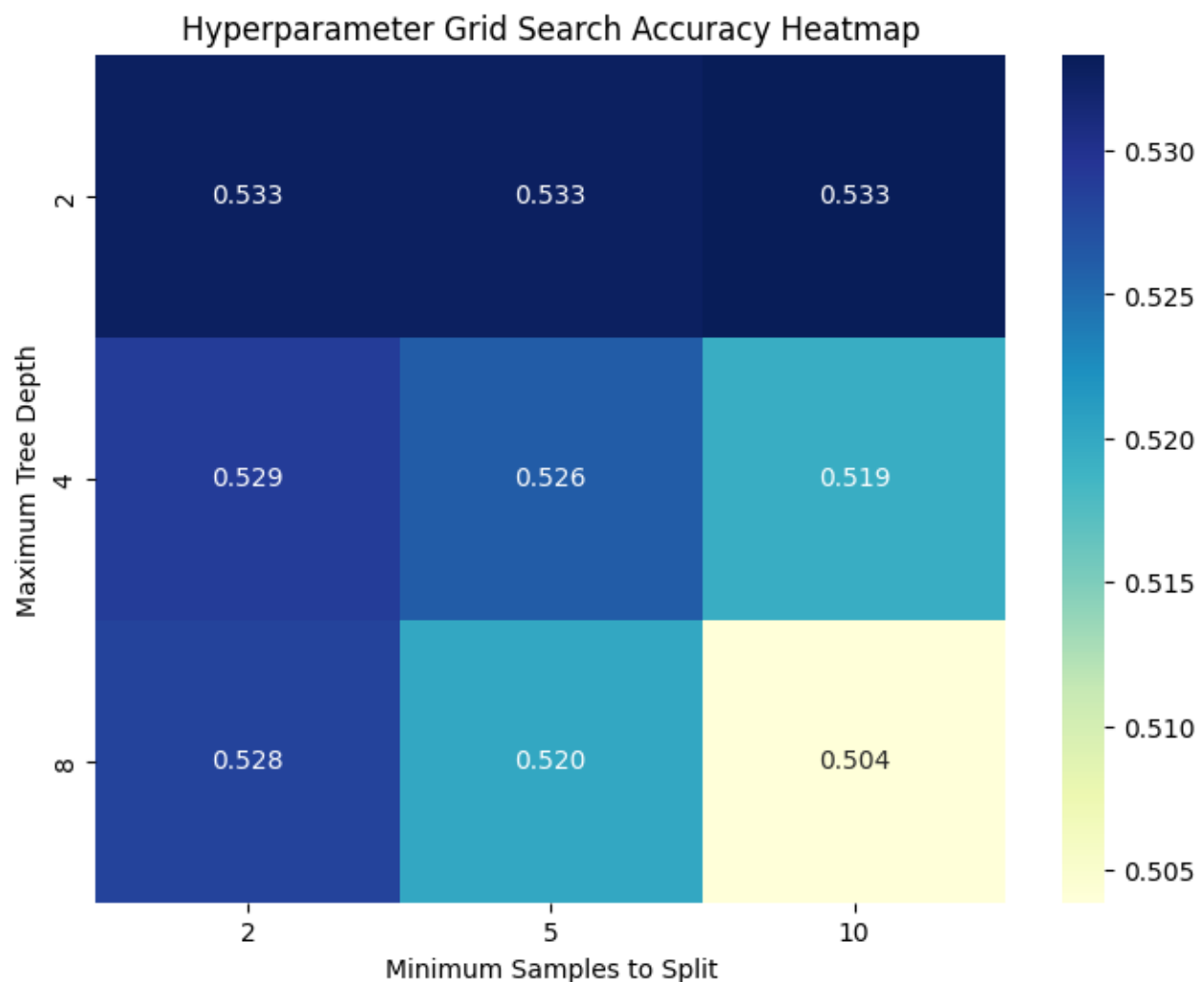
Analysts use automated search procedures to identify hyperparameter combinations that minimize forecasting errors (Yu and Zhu 3).

- **Grid Search:** This is a brute-force approach. You map out a discrete grid of values, and the algorithm tests every single combination. It is thorough but gets computationally massive very quickly.
- **Random Search:** Instead of checking every grid node, this method samples random combinations from a statistical distribution. Tests regularly show it finds the global optimum faster than Grid Search because it explores a wider variety of continuous numbers.
- **Bayesian Optimization:** This route looks at past training runs to predict the best next step. Using Gaussian Processes or Tree Parzen Estimators, it actually builds a probability model to hunt for the lowest expected error over time.

Validation Protocols:

Standard k-fold cross-validation is generally inappropriate for financial time-series data because randomly shuffling observations disrupts the temporal structure and introduces look-ahead bias. To fix this, we used TimeSeriesSplit. It acts as a walk-forward cross-validation tool that keeps the timeline intact so the model never accidentally trains on future events. The training window expands sequentially, and the model always validates on the next unseen block of time. This increases confidence that the tuning procedure will generalize more reliably to a live trading environment.

Outputs:



We ran both Grid and Randomized searches through a 5-fold Time Series Split. The Grid Search found our best setup: capping tree depth at 8, requiring 2 samples for a split, and using 10 estimators. This strict tuning yielded a **47.11%** out-of-sample accuracy on our test partition. While grabbing a sub-50% score highlights just how noisy financial time-series data really is, our walk-forward optimization did its job and kept the model from aggressively overfitting. The heatmaps visualize exactly how different depth constraints shifted the variance.

Non - Technical

Explanation of Results:

Deploying live trading systems requires careful calibration so the algorithm doesn't just memorize the past while ignoring the future. By running thousands of potential setups through a strict historical simulation, we found the precise guardrails that give the system its best shot at predictive stability. This keeps the program from being too rigid to spot new trends, while stopping it from trading recklessly on random market noise.

Recommended Action:

A structured recalibration schedule should be implemented for the execution desk. Market conditions evolve over time, meaning that a configuration calibrated during a bullish period may no longer remain effective during a recessionary environment. The overarching rules for all forecasting tools should automatically re-optimize at the end of every quarter, or right after major macroeconomic shifts.

Factors Impacting the Portfolio:

Our system's calibration is highly sensitive to the broader economy. If market volatility spikes or asset liquidity suddenly dries up, the system's risk-tolerance thresholds have to be tightened manually so it doesn't mistake a temporary panic for a sustainable trend.

Issue 2: Optimizing the Bias-Variance Tradeoff

(Author: Israel Olayinka Shodiya | Reviewer: Robert Nkuna)

Technical

The Tradeoff Mechanics:

The generalization error of a predictive model can be expressed mathematically as: $TotalError = Bias^2 + Variance + IrreducibleError$ (James et al. 340). Bias is the error you get from underfitting, when the model is too simple to catch real market relationships. Variance is the opposite; it measures how wildly the model reacts to tiny fluctuations in the training data, leading to overfitting.

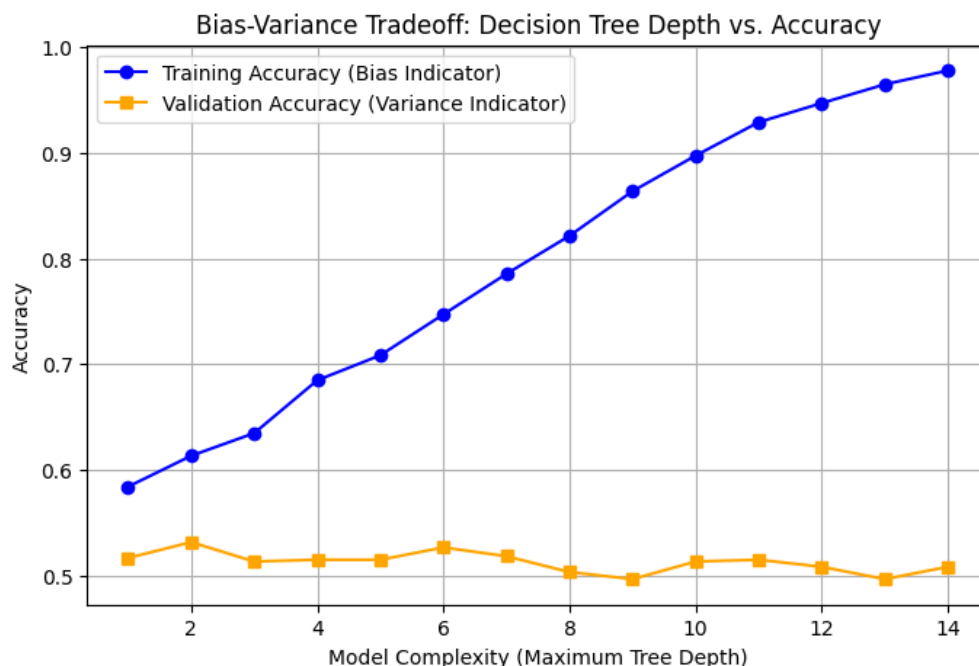
Model Complexity:

Bias and variance move in opposite directions as model complexity changes. Add more complexity (like deeper trees), and your bias drops while your variance explodes (James et al. 341). Hyperparameter optimization seeks to identify the point at which total error is minimized before the increase in variance outweighs the reduction in bias.

Optimization Techniques:

To manage this tradeoff, analysts commonly use techniques such as L1/L2 regularization to penalize massive coefficients, prune back decision trees, or trigger early stopping in neural networks the second validation errors start climbing.

Outputs:



The validation curve clearly illustrates this tradeoff on the SPY dataset. At a shallow max_depth of 1, the model suffers from massive bias and scores poorly across the board. As depth increases further, training accuracy rises toward 100% because the model starts memorizing the data. Validation accuracy, however, peaks around a depth of 2 and then steadily falls off. This divergence indicates that increasing complexity beyond depth 2 raises variance and reduces out-of-sample predictive performance.

Non - Technical

Explanation of Results:

A reliable forecasting tool has to walk a fine line. Make it too rigid, and it misses actual market trends. Make it too flexible, and it treats every random historical price jump like a permanent rule. Our testing proves that letting the system get too complex causes a massive drop in real-world performance.

Recommended Action:

Portfolio strategists should strongly favor conservative, generalized forecasting tools. Models should be deliberately constrained with strict complexity limits so they focus entirely on broad market directions instead of chasing historical anomalies.

Factors Impacting the Portfolio:

High-variance models can be particularly risky in live portfolios. Normal economic noise like an earnings report missing consensus by a fraction of a percent, can easily trick an over-engineered system into seeing a false pattern and executing highly leveraged, unwarranted trades.

Issue 3: Applying Ensemble Learning

(Author: Robert Nkuna | Reviewer: Rhushabh Vaghela)

Technical

Homogeneous vs. Heterogeneous Ensembles:

Ensemble learning improves predictive stability by combining several base estimators (Maclin and Optiz 169). Homogeneous versions stack up the exact same type of algorithm, like using dozens of decision trees inside a Random Forest. Heterogeneous ensembles combine different model architectures in order to benefit from complementary strengths (like mixing Trees, Support Vector Machines, and Logistic Regressions) so they can feed off each other's mathematical strengths.

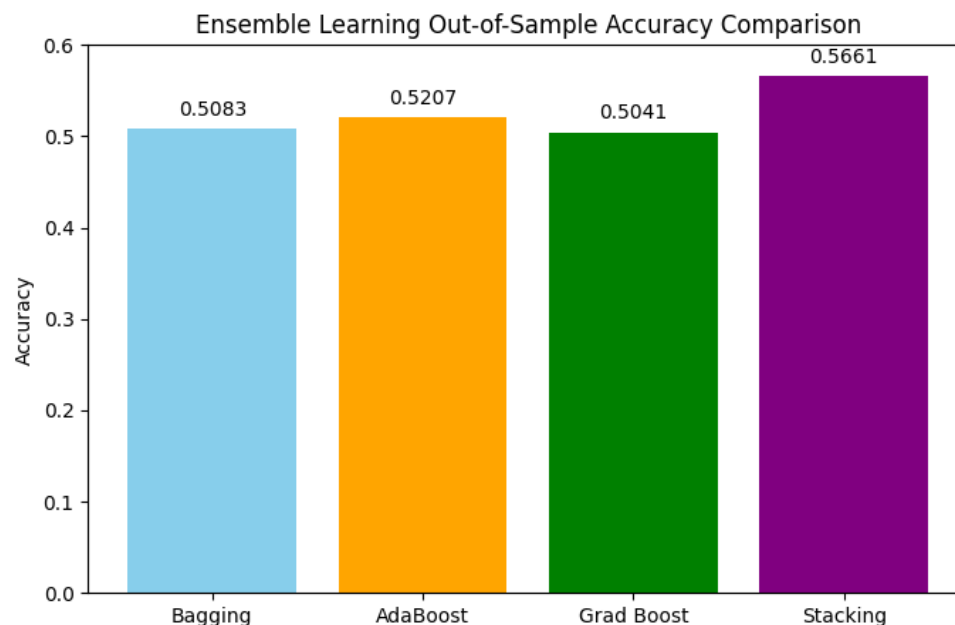
Bagging (Bootstrap Aggregating):

This method is designed primarily to reduce variance. It trains multiple models independently using random, replaced subsets of the training data. To get the final prediction, the framework simply averages the results through a majority vote: $\hat{y} = \frac{1}{M} \sum_{m=1}^M f_m(x)$ (Maclin and Optiz 170).

Boosting:

Boosting is designed primarily to reduce bias. It trains models in a sequence where every new model is mathematically forced to correct the residual errors made by the one right before it (Chuan et al. 2).

Outputs:



We evaluated Bagging, AdaBoost, Gradient Boosting, and Stacking classifiers to the test on the SPY dataset. The notebook's bar chart lays out the results. Because this feature set is so noisy, the Stacking classifier (a heterogeneous mix of a Decision Tree, SVM, and a Logistic Regression meta-model) achieved the strongest out-of-sample performance, with an accuracy of 56.61%. This suggests that combining distinct modeling perspectives can improve generalization relative to relying on individual base learners.

Non - Technical

Explanation of Results:

Ensemble learning works exactly like managing capital with a diverse committee of experts. If you trust just one analyst, you are vulnerable to their blind spots. Our testing proves that forcing different systems to analyze market data in entirely unique ways and using a chief decision-maker to weight their opinions gives us a much safer, more accurate market forecast.

Recommended Action:

A unified, ensemble-based forecasting system should be adopted for future capital allocation decisions. Standalone models should play a more limited role where ensemble methods demonstrate consistently stronger performance.

Factors Impacting the Portfolio:

Relying on a single predictive metric creates meaningful portfolio risk. By combining different viewpoints through ensemble models, the portfolio gets a layer of natural insulation. If one indicator mathematically fails during a liquidity crisis, the rest of the committee is there to stabilize the decision.

Technical Section - General Hyperparameter Tuning Strategy

Preparing a machine learning model for live deployment requires rigorous hyperparameter tuning in order to manage the bias-variance tradeoff. Hyperparameters act as external controls that govern how the algorithm learns.

To tune these globally, we define a hyperparameter space and deploy automated searches like Grid Search for rigid discrete testing, or Randomized Search for continuous testing to find the best possible combination. In quantitative finance, we have to validate these combinations using Walk-Forward Cross-Validation (TimeSeriesSplit). By forcing the validation to respect the timeline chronologically, we guarantee that future data never leaks into the training folds. This is necessary to improve confidence that the selected parameters will remain effective under unseen market conditions.

Examples of Hyperparameters Tuned in Our Individual Reports:

- **Decision Trees (Issue 1 & 2):** We tuned `max_depth` to limit tree growth, stopping them from absorbing high-variance noise. As shown in Issue 2, hard-capping the depth at 2 perfectly balanced the bias and variance. We also optimized `min_samples_split` (landing on 2 or 12 depending on the search tool) to require strict data thresholds before allowing a node to branch.
- **Ensembles (Issue 3):** For both Bagging and Boosting, we tuned `n_estimators`. This essentially sets the exact size of our "committee" (for example, we locked in 50 distinct estimators for the AdaBoost model).
- **Support Vector Machines:** When we plugged SVMs into our heterogeneous Stacking ensemble, we used the regularization penalty `C` to dictate exactly how strict the decision boundary margins needed to be.

Marketing Alpha - The Machine Learning Advantage

Beyond Theoretical Models to Live Market Resilience:

The previous report established the theoretical foundation for the machine learning models considered in this project. However, practical value in quantitative finance depends not only on model selection, but also on calibration, robustness, and out-of-sample reliability. The current framework was therefore refined to better address the noise and instability characteristic of financial markets.

Capital Protection through Controlled Complexity:

Capital preservation depends on controlling model complexity and managing the bias-variance tradeoff carefully. Overly complex models may fit historical anomalies rather than persistent market structure, which can weaken performance when market regimes change. By applying disciplined complexity constraints, the models are better positioned to capture broader and more stable patterns.

The "Committee of Experts" Approach:

Ensemble learning provides an effective way to combine complementary model perspectives. By integrating decision trees, support vector machines, and logistic regression within a stacking framework, the system benefits from diversification in model structure and decision logic. This contributes to greater robustness when individual indicators become less reliable during periods of market stress.

True Out-of-Sample Confidence:

The reported performance metrics are supported by validation procedures that preserve the chronological structure of the data. By using walk-forward style validation, the analysis reduces look-ahead bias and provides a more credible indication of how the models may behave in forward-looking settings.

References:

Blanco, Víctor, et al. "Multiclass Optimal Classification Trees with SVM-splits." *arXiv*, 2021, pp. 1-8.

Chuan, Yijian, et al. "The Success of AdaBoost and Its Application in Portfolio Management." *arXiv*, 2021, pp. 1-5.

Ho, Winky K.O, et al. "Predicting Property Prices with Machine Learning Algorithms." *Journal of Property Research*, vol. 38, no. 1, 2021.

James, Gareth, et al. *An Introduction to Statistical Learning with Applications in R*. 2nd ed., Springer, 2021, pp. 327-382.

Maclin, R., and D. Optiz. "Popular Ensemble Methods: An Empirical Study." *Journal of Artificial Intelligence Research*, vol. 11, 1999, pp. 169-198.

Murphy, Kevin Patrick. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022, pp. 609-632.

Yu, Tong, and Hong Zhu. "Hyper-Parameter Optimization: A Review of Algorithms and Applications." *arXiv*, 2020.